Research Updates 10 to 13: Team Summary

July 5 to August 1, 2026. Four work cycles, about 80 hours.

What this is about

The model detects chatter in milling. It was built around a **delay**: the time one tooth takes to reach the spot the previous tooth just cut. Chatter starts when a tooth bites into the wavy surface the last tooth left behind, so that delay is the physics of the problem.

Earlier cycles showed the model does use the delay. These four cycles asked two new questions. Does that explanation survive when I attack it? And can I add a new kind of evidence for it?

It survives. And no: four attempts at new evidence all failed, and I report each one as a failure.

Update	Dates	Question	Answer
10	Jul 5-11	Does the explanation survive noise and damage?	**Yes.**
11	Jul 12-18	Can I finish the remaining explanation tools?	**Three worked, two failed.**
12	Jul 19-25	Does saliency give new delay evidence?	**No.** Broken measurement; the fix found an artifact.
13	Jul 26-Aug 1	Does the advice survive shop-floor tolerances? Did the model learn the textbook chatter term?	**Partly. And no.**

Throughout, **detection score** means how cleanly the model separates chatter from clean cuts: 1.0 is perfect, 0.5 is a coin flip.

Update 10: I tried to break the model, and it held

I damaged the model and its inputs on purpose, in graded steps, five different ways: added noise, blurred the signal, corrupted the delay value, added noise to the model's own weights, and coarsened its internal curves. At every step I recorded two numbers — how well it still detected chatter, and how much it still leaned on the delayed input. Detection holding up while the delay reliance collapsed would mean the model had quietly switched from physics to memorisation.

That never happened. Everything degrades gracefully. Noise at twice the strength of the signal leaves the detection score above 0.75. Adding 50 percent random noise to the weights costs almost nothing, so the solution sits in a wide valley rather than balanced on a knife edge.

The most informative attack corrupts the delay alone: same signal, same weights, only the delay value wrong. Detection then falls off smoothly, in proportion to how wrong the delay is. Because nothing else changed, that is cause and effect rather than correlation. It repeats on all five separately trained copies of the model.

Update 11: three tools worked, two failed

All five ran on models that were already trained. Nothing was retrained.

Worked — reading the learned curves. The model is built out of small learned curves. I ranked the 25 most important ones and fitted a plain formula to each. Every one accepted a formula, most of them near-exactly. Curves fed by the delayed input hold about 15 percent of the total importance on three of the five models and 3 to 4 percent on the other two — the same split every other test shows.

Worked — turning the model into advice. Given a cut the model calls chatter, what is the smallest change that makes it call the cut stable? The search proposes a new spindle speed and depth. Its answers agree with the textbook stability boundary 82 percent of the time. An earlier version scored 95 percent, but it had been fitted to the dataset's labels instead of to the model's own predictions, which made agreement almost automatic. The lower number is the honest one. The advice is almost always to change spindle speed and leave the depth alone: 35 of the 38 answers move speed only, usually downward, toward the nearest stable window.

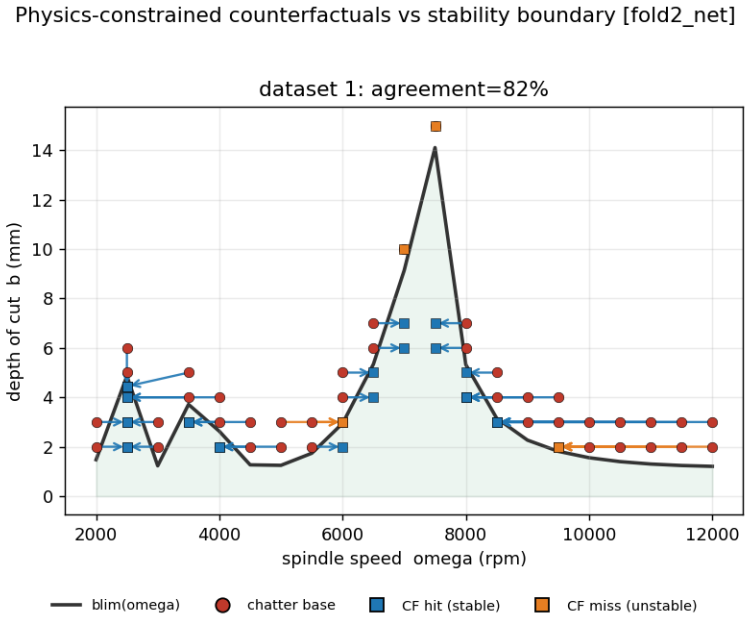


Figure 1: The model's advice, drawn on the machinist's map. The horizontal axis is spindle speed, the vertical axis is depth of cut. The black curve is the textbook stability limit: cuts below it are safe, cuts above it chatter. Red dots are chattering setups; each arrow is the smallest change the model asks for. Blue squares land in genuinely safe territory, orange squares satisfy the model but stay physically unsafe.

Worked — confidence and the option to abstain. Every prediction now carries a confidence score, and the model may decline to answer. When it does answer, it is right 99.6 percent of the time, against 98.4 percent when forced to answer everything.

Failed — the concept probe. I tried to read a specific physical quantity out of the model's internal state. It fits the training data and fails completely on held-out data, so the quantity is simply not there in readable form. Claim dropped. One related quantity — how close a cut sits to the stability limit — does survive.

Failed — saliency. This method is meant to give credit to inputs that act through long delays, which simpler attribution methods under-weight. It ran, and it put no peak at the delay. Update 12 explains why.

Update 12: a dead end, and the bug behind it

The measurement was broken. The routine that hunts for the delay in a saliency map returns the bottom of its search range when it finds nothing at all. That "no answer" value is indistinguishable in the output from a real answer of the same number. 550 of the 640 windows returned it. Every per-model average was therefore an average over a mostly empty column. So my earlier diagnosis — "not enough data" — was wrong. The measurement was unsound.

I rebuilt it, found a signal, then killed it. The new version tests directly for repetition at the delay, so it has no floor to fall back on. Of the 90 windows that gave a real answer, 31 landed exactly on their own delay, against 3 expected by chance. That looked like a genuine result. Then I ran the control: the same network with random, untrained weights.

Network	Strength of the lock onto the delay	Windows that lock
Trained, correct delay	+2.81	79 percent
Untrained, correct delay	**+4.62**	**100 percent**
Trained, wrong delay	+0.88	17 percent

The untrained network locks on harder than the trained one. The lock therefore comes from how the input is built — the delayed channel is literally the signal shifted by that window's delay — and not from anything the model learned. Two further checks agree. The verdict rule was fixed in advance of the run. Saliency is now a closed negative and stays out of the technical report.

Failed — the frequency-notch test. The idea was to delete the chatter frequency band from the signal, delete the same amount of energy from the tooth-passing frequencies, and see which hurts the model more. The two cannot be matched. The chatter band is narrow, while the tooth-passing peaks are among the strongest content in the whole signal, so the smallest cut you can make to one is far below the smallest cut you can make to the other. The two ranges never overlap. Widening the chatter band to close the gap makes it swallow the tooth-passing peaks, which deletes both conditions at once. About 60 percent of windows are unsolvable on every model, and both obvious workarounds left the gap two to four times too wide. This one is structural, not a tuning problem.

Two retrainings, both null. Feeding the process parameters (speed, depth, feed) into the model changed nothing measurable. Changing how the model predicts chatter severity changed nothing measurable. Both effects land below the run-to-run noise floor, so neither is claimable.

Update 13: does the advice survive a real shop floor?

Tolerance test. The 82 percent agreement describes an exact operating point, which no machinist ever hits. So I re-ran every recommendation 512 times with realistic slop: 2 percent on spindle speed, 0.05 mm on depth, and 10 percent uncertainty on the stability boundary itself.

19 of the 42 recommendations stay stable in more than 85 percent of the draws. The bar set in the plan was 10, so the criterion passes. But the median recommendation survives only 69 percent of the time and the worst survives 9 percent. Most of this advice is not yet something to hand a machinist.

The 10 percent boundary uncertainty drives that result, and it is my judgement call rather than a measurement. It belongs next to the headline every time the headline is quoted.

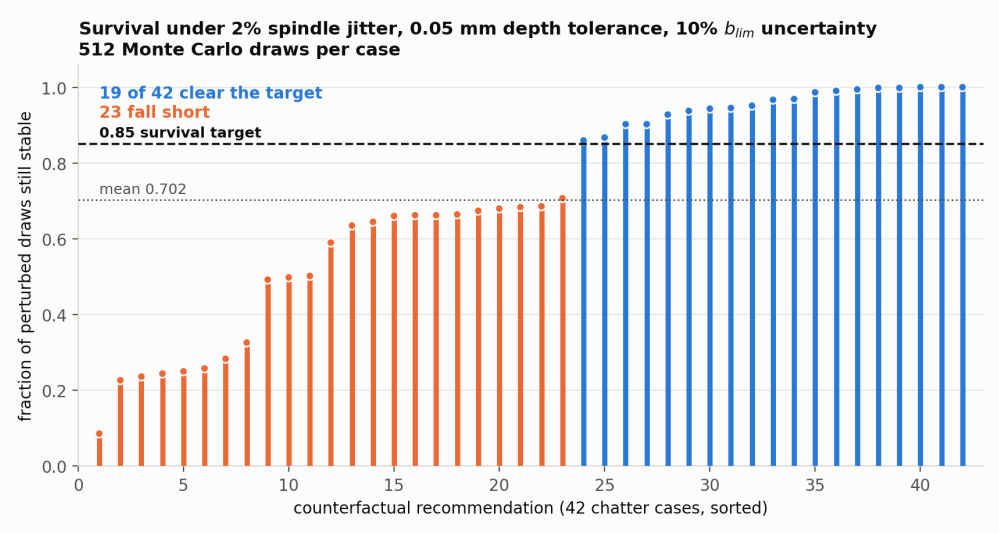


Figure 2: How much of the advice survives real-world slop. Each bar is one recommendation; its height is the fraction of 512 perturbed repeats that stayed safe. Blue bars clear the 85 percent target, orange bars fall short.

A second opinion. I ran an off-the-shelf tool (DiCE) that never sees the stability boundary at all. The two methods agree on the outcome in 27 of 42 cases. They get there by different routes: mine mostly changes spindle speed, DiCE mostly cuts depth. My first scoring counted that as disagreement, which was wrong — my search is forbidden from increasing depth, so it could never have chosen DiCE's route. Which knob gets turned is now reported as a description, not scored.

Did the model learn the textbook chatter term? No. In the textbooks, regeneration is a *difference*: the surface now, minus the surface one tooth ago. Inside the model every delayed input has an instantaneous twin, so a model that learned that difference should show the delayed curve as roughly the mirror image of its instantaneous twin.

The opposite is true. On all five models the delayed curve is close to a *copy* of its twin, not a mirror image, and about thirty times more pairs add than subtract. The controls behave as they should: untrained models show nothing, and unrelated pairs show nothing.

So the model is not computing the textbook difference. The pattern looks like a sum, which would reinforce tooth-synchronous content instead. That reading is plausible but unproven — this test inspects individual curves, not the full network.

Where this leaves the project

The delay claim is unchanged. Four separate results support it, and not one of them is a saliency map:

1. Zero out the delayed input and detection collapses to near chance.
2. Sweep the delay across a window and the best value is the real tooth-passing delay.
3. Corrupt the delay and accuracy degrades in proportion.
4. A completely separate method with no neural network in it (SINDy-DDE) recovers the same delay.

Four things died this month. Saliency — an artifact of how the input is built. The concept probe — the concept is not there. The frequency-notch test — impossible to set up fairly. The textbook-difference test — the model sums instead.

One mistake, four times over. Every one of these failures had the same shape. A routine hits a limit, returns a placeholder, and that placeholder flows into an average that cannot tell a placeholder from a measurement. A search floor read as a real answer. A notch solver that gave up and returned a band removing exactly zero energy. Two cluster jobs that exited successfully in 90 seconds without running anything.

Three standing rules now: placeholders never enter an average; any number computed from a subset reports how big that subset was; and every job wrapper must confirm its output exists before exiting cleanly.

Methods used across the four cycles: graded damage to inputs and model, formula fitting on the learned curves, counterfactual search, DiCE, calibrated confidence, linear concept probes, saliency with a random-weight control, frequency-band deletion, Monte Carlo tolerance propagation, and network-free SINDy-DDE delay recovery.